

PROJECT REPORT



AI Real Estate Intelligence Platform

Price prediction · Explained recommendations · AI advisor · Document RAG

Submitted by

Anushay Ayat

Program

Artificial Intelligence and Data Science

Institute

SMIT (Saylani Mass IT Training)

Date

October 2026

Abstract

PropMind is an AI-powered real estate intelligence platform that analyzes properties, predicts prices, recommends homes with explanations and answers questions through a generative AI advisor and a document-grounded RAG assistant. It is built on the Ames Housing dataset (1,458 cleaned home sales) and delivered as a FastAPI application with a premium web interface.

After cleaning and exploratory analysis, three regression models were compared. The deployed Random Forest reaches an R^2 of 0.885 and a mean absolute error of \$17,782 on unseen homes, with 77% of predictions within 15% of the real price. A transparent recommender ranks homes by budget fit, value against the AI fair price, quality and preferences and explains every result. The assistant uses a structured prompt with injected market facts and safety rules, and degrades gracefully to an offline mode. The RAG assistant retrieves passages from four property documents with stemmed TF-IDF and answers with citations, ranking the correct document first for all 15 evaluation questions.

Testing the system as a product exposed and fixed real defects, including a neighborhood feature that the first model ignored. The project includes automated tests, a complete notebook, deployment configuration for GitHub, Vercel and Firebase, and full documentation.

Keywords: real estate, price prediction, Random Forest, recommendation system, generative AI, RAG, FastAPI, prompt engineering

Table of Contents

1. Introduction.....	4
1.1 Background and Motivation.....	4
1.2 Problem Statement.....	4
1.3 Objectives.....	4
1.4 Assignment Requirements and Where They Are Met.....	5
2. Dataset and Preprocessing.....	6
2.1 Source and Description.....	6
2.2 Missing Values.....	6
2.3 Cleaning and Feature Engineering.....	6
2.4 Features Used by the Deployed Model.....	7
3. Exploratory Data Analysis.....	8
3.1 Price Distribution.....	8
3.2 Location Comparison.....	8
3.3 Size and Quality.....	9
3.4 Age, Trend and Correlations.....	9
4. Machine Learning Methodology.....	11
4.1 Problem Formulation.....	11
4.2 Pipeline.....	11
4.3 Models Compared.....	11
4.4 Evaluation Protocol.....	11
4.5 A Defect Found Through Testing.....	11
5. Results and Evaluation.....	13
5.1 Model Comparison.....	13
5.2 How Close Are the Predictions?.....	13
5.3 Feature Importance.....	14
5.4 What-If Analysis.....	15
6. Recommendation System.....	16
6.1 Inputs and Filtering.....	16
6.2 Scoring.....	16
6.3 Explanations and Worked Example.....	16
7. Generative AI Assistant.....	17
7.1 Design.....	17
7.2 Prompt Engineering.....	17
7.3 Property Summaries.....	17
7.4 Reliability: Offline Mode.....	17
8. RAG Knowledge Assistant.....	19
8.1 Knowledge Base.....	19
8.2 Pipeline.....	19
8.3 Grounded Generation.....	19
8.4 Retrieval Evaluation.....	20
9. System Design and Implementation.....	21
9.1 Architecture.....	21
9.2 Backend Modules.....	21
9.3 API Endpoints.....	22

9.4 Web Interface.....	22
9.5 Additional Features.....	25
10. Testing and Quality Assurance.....	26
10.1 Automated Tests.....	26
10.2 Browser Tests.....	26
10.3 Defects Found and Fixed.....	26
11. Deployment and Operations.....	27
12. Limitations and Future Work.....	28
12.1 Limitations.....	28
12.2 Future Work.....	28
13. Conclusion.....	29
14. References.....	30
Appendix A: Setup and Run Guide.....	31
Appendix B: Example API Call.....	32

1. Introduction

1.1 Background and Motivation

Buying a home is the largest financial decision most people make, yet the information available to them is scattered and hard to trust. Listing prices are opinions, neighborhood comparisons live in spreadsheets, and the legal and financial rules of a purchase are buried in long documents. Recent advances in machine learning and generative AI make it possible to bring these pieces together in a single, explainable product.

PropMind is an AI-powered real estate platform built to demonstrate that idea end to end. Rather than stopping at a trained model in a notebook, it delivers a complete product: a data pipeline, a price-prediction model, an explainable recommendation engine, a generative AI advisor, a retrieval-augmented generation (RAG) assistant over property documents, a FastAPI backend and a premium web interface.

1.2 Problem Statement

A prospective buyer faces four connected problems:

- **Opaque pricing:** it is hard to know whether an asking price is fair for a specific home.
- **Manual matching:** filtering listings by budget, location and preferences is slow, and tools rarely explain why a home is a good fit.
- **Hard-to-read documents:** buying guides, legal procedures and investment advice are long and difficult to search.
- **Fragmented tools:** valuation, search, advice and document help sit in separate places.

1.3 Objectives

1. Clean and analyze a real housing dataset and communicate its main patterns visually.
2. Train, compare and evaluate machine-learning models that predict sale prices, and deploy the best one.
3. Recommend properties from a user's budget, location, bedrooms and preferences, and explain every recommendation.
4. Provide a generative AI assistant with careful prompt engineering and safe behavior.
5. Answer questions from property documents using RAG, with citations to the exact passages.
6. Package everything as a clean, deployable web application with tests, documentation and source control.

1.4 Assignment Requirements and Where They Are Met

Table 1: Mapping of assignment requirements to this report and the project

#	Requirement	How PropMind meets it	Section
1	Data analysis	Cleaning and preprocessing with Pandas and NumPy; eight Matplotlib and Seaborn charts (price, location, area vs price, quality, trend, correlations)	2, 3
2	ML price prediction	Linear Regression, Decision Tree and Random Forest compared; best model evaluated and saved with joblib	4, 5
3	Recommendation system	Filters by budget, location, bedrooms and preferences; every result explains why it was recommended	6
4	AI real estate assistant	Groq (Llama 3.3) chatbot with prompt engineering, market trends, property summaries and buying guidance	7
5	RAG knowledge assistant	Four property documents plus user uploads; answers use only retrieved passages and cite them	8
6	FastAPI and web interface	Prediction, recommendation and chatbot pages, plus insights, documents and about pages	9
7	GitHub submission	Source code, dataset, requirements, README, screenshots, notebook and tests	11, App. A

2. Dataset and Preprocessing

2.1 Source and Description

The project uses the Ames Housing dataset, compiled by Dean De Cock (2011) and distributed through Kaggle's House Prices: Advanced Regression Techniques competition under the MIT license. The training file contains *1,460 home sales in Ames, Iowa, recorded between 2006 and 2010, described by 79 explanatory variables* covering size, quality, age, location, garage, basement and many other features, plus the target SalePrice in US dollars.

The dataset is rich and well documented, which makes it a good teaching resource. Its limits are equally important: it describes a single city and a past period, so the resulting estimates are educational and not suitable as appraisals.

2.2 Missing Values

19 of the 81 columns contain missing values, about 7,829 empty cells in total. Most of these are not genuine gaps. In columns such as PoolQC, Alley, Fence or the garage and basement descriptors, "NA" means that the feature does not exist (no pool, no alley, no garage). Treating them as missing and dropping or imputing them would destroy real information, so they are encoded as an explicit "None" category instead.

2.3 Cleaning and Feature Engineering

Table 2: *Cleaning steps applied to the raw data*

Step	Action	Reason
1	Drop the Id column	Identifier with no predictive meaning
2	Fill 15 descriptor columns with "None"	NA means the feature is absent
3	Impute LotFrontage with the neighborhood median	Homes in one neighborhood have similar lot frontage
4	Set GarageYrBlt to YearBuilt; MasVnrArea to 0; Electrical to the mode	Sensible defaults for the few remaining gaps
5	Remove two outliers (living area over 4,000 sqft sold under \$300,000)	Documented anomalies that distort the area-price relationship
6	Create TotalSF, HouseAge and PricePerSqft	Summarize size and age; PricePerSqft supports neighborhood comparison

After cleaning, the data contains **1,458 homes and 83 columns with no missing values**.

2.4 Features Used by the Deployed Model

To keep the web form simple enough for a real user, the deployed model uses nine inputs that a buyer can know without technical effort:

Table 3: *Model inputs*

Feature	Meaning	Type
GrLivArea	Above-ground living area (sqft)	Numeric
BedroomAbvGr	Bedrooms above ground	Numeric
FullBath	Full bathrooms	Numeric
OverallQual	Overall material and finish quality (1-10)	Ordinal
YearBuilt	Original construction year	Numeric
GarageCars	Garage capacity in cars	Numeric
TotalBsmntSF	Total basement area (sqft)	Numeric
LotArea	Lot size (sqft)	Numeric
Neighborhood	One of 25 Ames neighborhoods	Categorical

3. Exploratory Data Analysis

3.1 Price Distribution

Sale prices are right-skewed: the mean (\$180,933) sits well above the median (\$163,000) because a small number of expensive homes pull it upward. The skewness of the raw price is 1.88; after a log transform it drops to 0.12, which is close to a symmetric distribution. This motivates predicting the logarithm of the price in the models (Section 4).

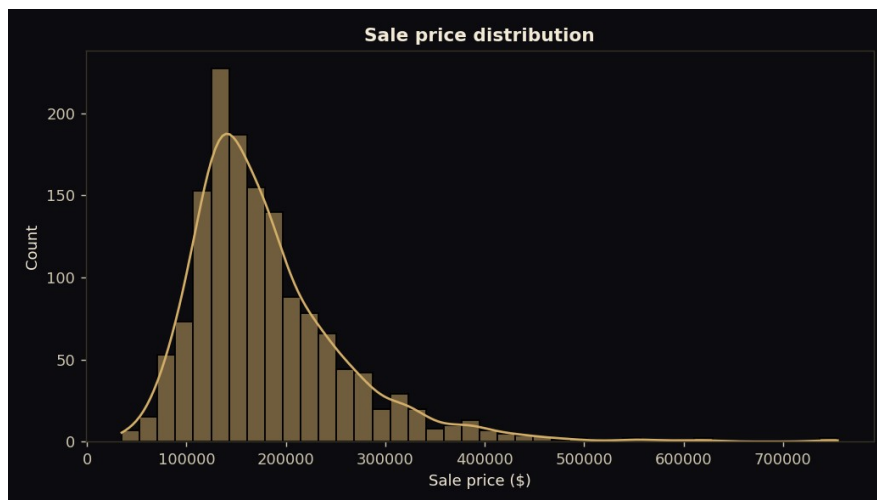


Figure 1: Distribution of sale prices

3.2 Location Comparison

Location matters strongly. The highest median prices are in NridgHt (\$315,000), NoRidge (\$301,500), StoneBr (\$278,000); the lowest are in BrDale (\$106,000), IDOTRR (\$103,000), MeadowV (\$88,000). The most expensive neighborhood, NridgHt, has a median price about 3.6 times that of the cheapest, MeadowV.

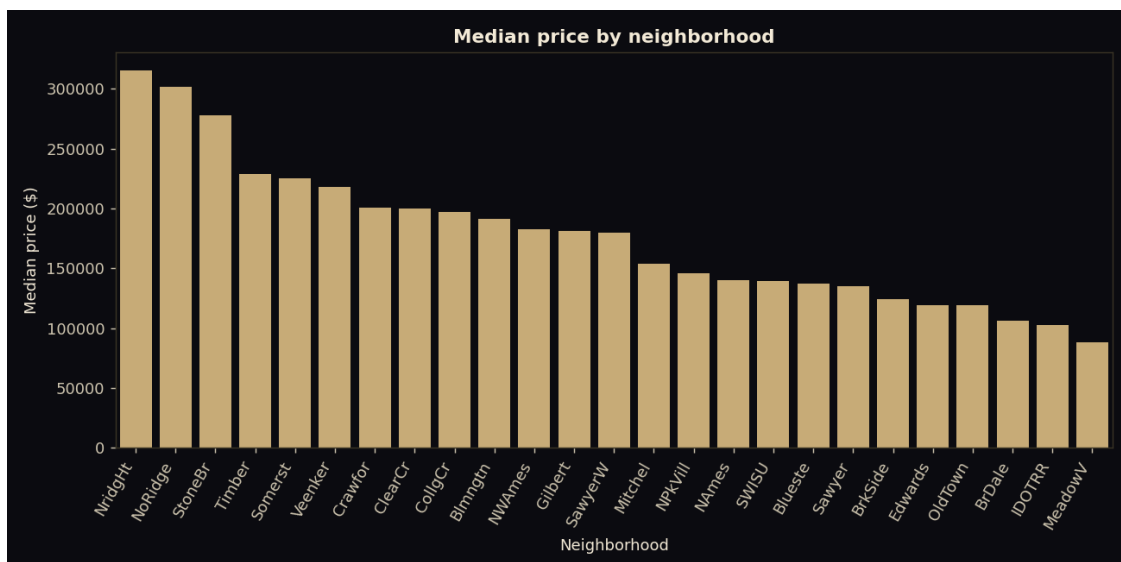


Figure 2: Median sale price by neighborhood

3.3 Size and Quality

Living area and quality are the strongest numeric drivers of price. The correlation between living area and price is 0.735, and between overall quality and price 0.796. Median price rises steeply with quality: \$86,250 at quality 3, \$133,000 at 5, \$200,141 at 7, \$345,000 at 9 and \$445,365 at 10.

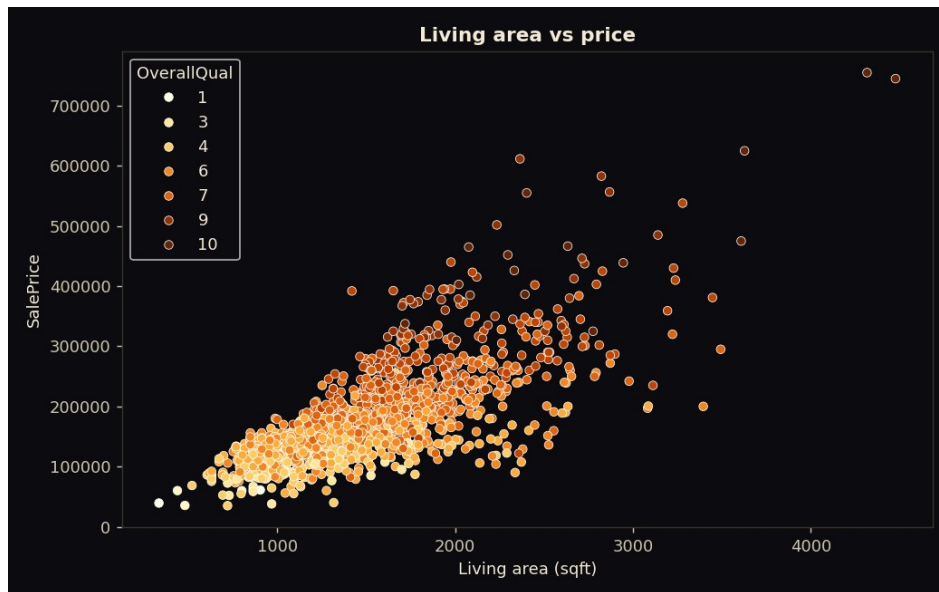


Figure 3: Living area versus sale price, colored by overall quality

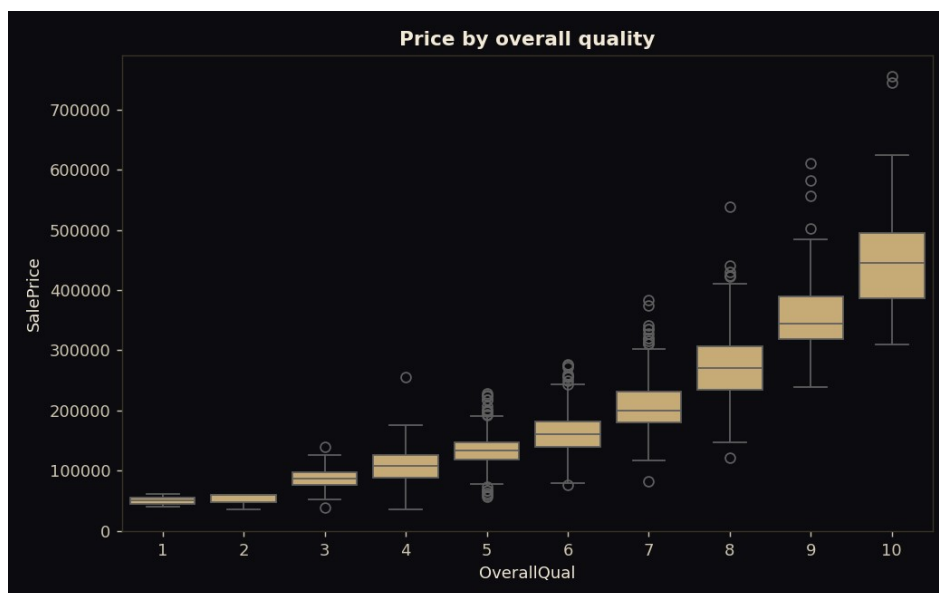


Figure 4: Sale price by overall quality

3.4 Age, Trend and Correlations

Newer homes tend to sell for more, although the effect is partly explained by quality and size. A correlation heatmap of the eleven features most related to price confirms that quality, area, garage capacity, basement size and year built move together with price.

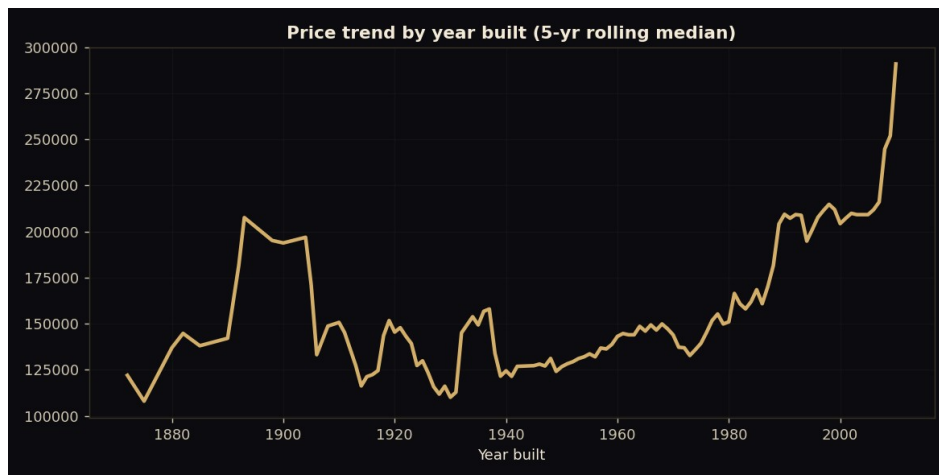


Figure 5: Median price by year built (five-year rolling average)

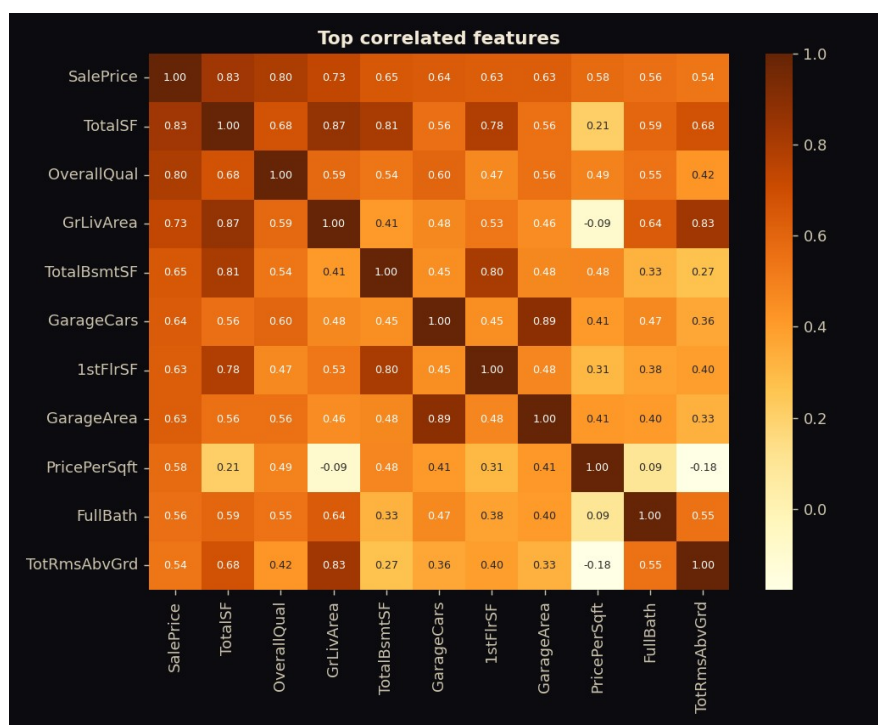


Figure 6: Features most correlated with sale price

Key insights. (1) Overall quality is the single strongest signal. (2) Living area and garage capacity add steady value. (3) Neighborhood adds a large premium or discount on top of physical features. (4) Prices are skewed, so medians describe a typical home better than means.

4. Machine Learning Methodology

4.1 Problem Formulation

Price prediction is a supervised regression problem. The target is $\log(1 + \text{SalePrice})$; predictions are converted back to dollars with the inverse transform. Working in log space stabilizes the variance, reduces the influence of very expensive homes and makes errors proportional to price.

4.2 Pipeline

A scikit-learn Pipeline combines preprocessing and the model so that exactly the same steps run during training and in the web application. The neighborhood is converted to a number with a **target encoder** (its smoothed average log price, computed with internal cross-fitting to avoid leakage), and the eight numeric features pass through unchanged. The fitted pipeline is saved as a single joblib file of about 2 MB.

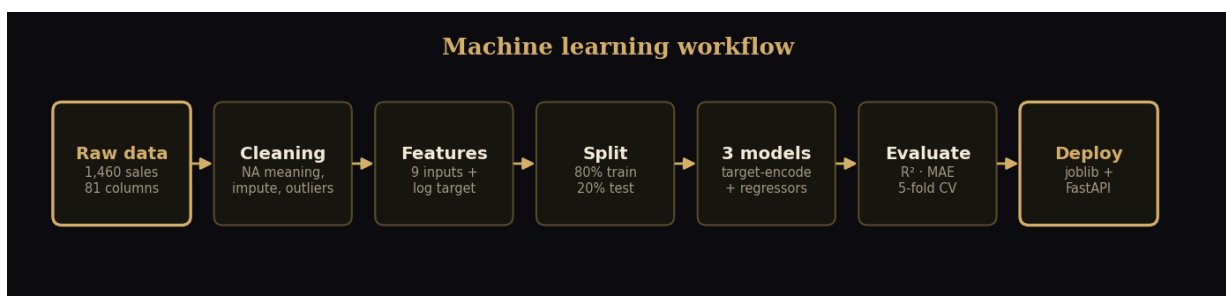


Figure 7: Machine-learning workflow

4.3 Models Compared

Table 4: Models and settings

Model	Settings	Why it is included
Linear Regression	Defaults	Simple, interpretable baseline
Decision Tree	max_depth 9, min_samples_leaf 5	Non-linear model that is easy to explain
Random Forest	150 trees, max_depth 16, min_samples_leaf 2	Ensemble that captures interactions and reduces variance

4.4 Evaluation Protocol

The data is split 80/20 with a fixed random seed (1,166 training and 292 test homes). Models are compared on the held-out test set using R^2 , mean absolute error (MAE) and root mean squared error (RMSE) in dollars, together with five-fold cross-validated R^2 on the training data. In the accompanying notebook, a grid search over tree count, depth and leaf size selects the Random Forest settings (best cross-validated R^2 0.8455; test R^2 0.884, MAE \$17,811).

4.5 A Defect Found Through Testing

The first version encoded the neighborhood with one-hot columns. Accuracy looked fine, but automated tests exposed a serious problem: for the same house, most neighborhoods returned an identical estimate of

\$203,204, so changing the neighborhood in the web form did nothing. The trees almost never split on the sparse one-hot columns, so the location signal was ignored.

Replacing one-hot encoding with target encoding fixed the problem. For the same home, estimates now range from \$165,473 to \$210,800 across neighborhoods (21 distinct values), the neighborhood contributes about 10% of the model's importance, and test performance improved slightly. This episode illustrates why a model should be tested as a product and not only scored with a metric.

5. Results and Evaluation

5.1 Model Comparison

Table 5: Test-set performance of the three models

Model	R ²	MAE	RMSE	5-fold CV R ²
Linear Regression	0.876	\$18,829	\$26,144	0.859
Decision Tree	0.840	\$20,913	\$29,761	0.769
Random Forest (deployed)	0.885	\$17,782	\$25,251	0.847

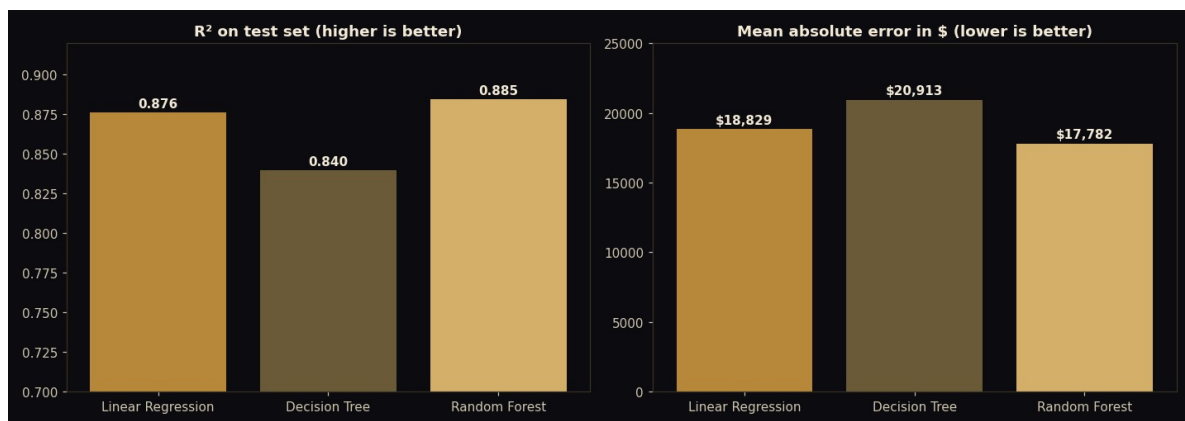


Figure 8: R² and mean absolute error for the three models

The Random Forest achieves the best test R² (0.885) and the lowest errors. Linear Regression is close behind and even has a slightly higher cross-validated R² (0.859 against 0.847), so the difference between the two is small. The Random Forest was selected because of its lower average error on unseen data and because it captures non-linear effects such as the steep premium for high build quality. The Decision Tree is clearly weaker, as expected for a single tree.

5.2 How Close Are the Predictions?

Table 6: Share of test predictions within a given error

Error band	Share of test homes
Within 10% of the real price	63%
Within 15%	77%
Within 20%	87%
Median absolute percentage error	6.6%

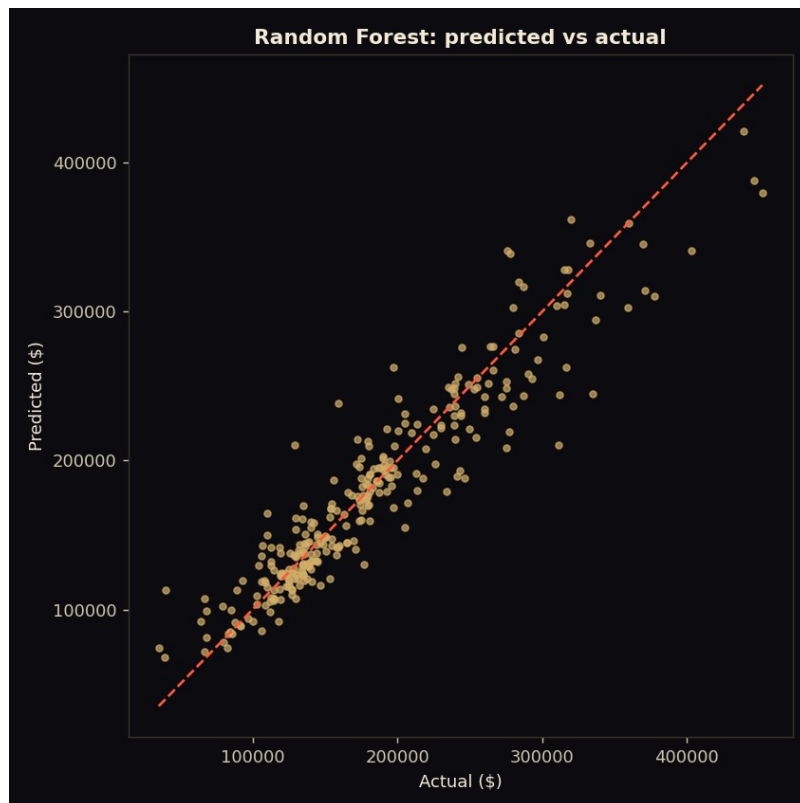


Figure 9: Predicted versus actual sale price on the test set

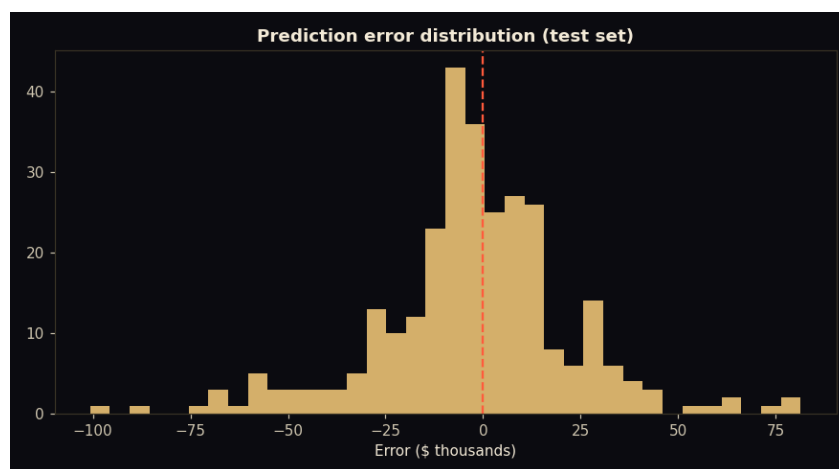


Figure 10: Distribution of prediction errors

Errors are centered on zero with no strong bias. The model is less accurate for the most expensive homes, which are rare in the data.

5.3 Feature Importance

Overall quality dominates (about 54% of total importance), followed by living area (15%), neighborhood (10%), basement size (8.5%), garage capacity (4.8%), year built (3.6%) and lot area (3.0%). Bedrooms and bathrooms contribute little once size and quality are known, which matches the exploratory analysis.

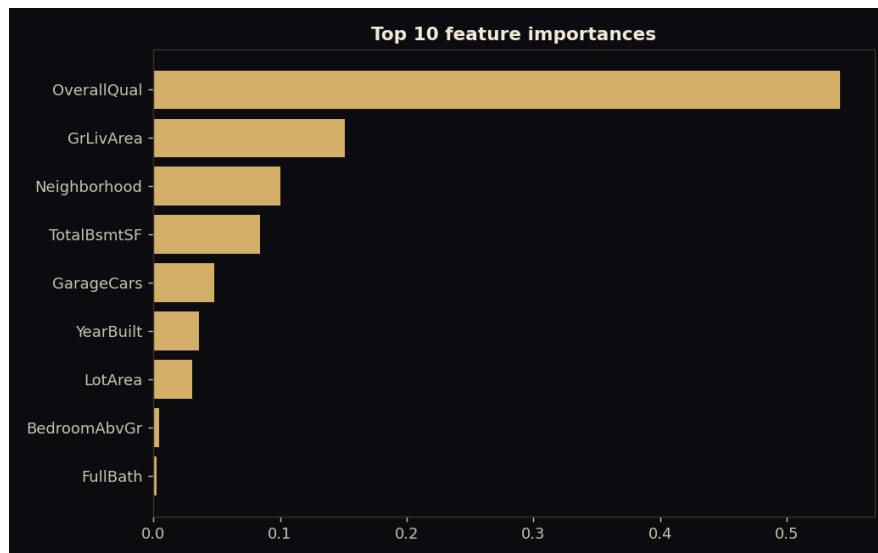


Figure 11: Top feature importances of the Random Forest

5.4 What-If Analysis

To make the model explainable, the application re-predicts the price after changing one feature. For a sample three-bedroom home in College Creek (estimate \$198,017), one extra quality point adds about \$17,616, 200 extra square feet add about \$11,123 and one more garage space adds about \$8,376. Because a forest is not a smooth function, such effects should be read as model estimates and not as exact renovation returns.

6. Recommendation System

6.1 Inputs and Filtering

The user provides a maximum budget, a minimum number of bedrooms, and optional neighborhoods and preferences (garage, newer build, high quality, large lot, spacious home, big basement). Homes outside the budget or bedroom requirement are removed first.

6.2 Scoring

Table 7: Components of the match score

Component	Weight	Meaning
Budget fit	30%	Homes that use the budget well score higher than very cheap ones
Value	30%	Sale price compared with the AI fair-value estimate; below the estimate scores higher
Build quality	20%	Overall quality rating divided by 10
Preferences	20%	Share of the chosen preferences the home satisfies (living area when none is chosen)

6.3 Explanations and Worked Example

For a budget of \$250,000, at least three bedrooms, the neighborhoods College Creek, Somerset and Gilbert, and the preferences garage and newer build, **216 homes** qualify. Every result carries plain-language reasons, which makes the ranking auditable. The top result is: a 3-bedroom home in CollgCr priced at \$245,350 with a match score of 83. Its reasons are:

- Priced at \$245,350, \$4,650 under your \$250,000 budget
- Listed 9% below the AI fair-value estimate of \$268,289
- 3 bedrooms, meeting your 3+ requirement
- Located in CollgCr, your preferred area
- Matches your preference: has a 3-car garage
- Matches your preference: built in 2008 (newer construction)

Table 8: Top three recommendations for the example query

Rank	Neighborhood	Price	AI fair value	Beds	Sqft	Quality	Built	Score
1	CollgCr	\$245,350	\$268,289	3	1,694	8/10	2008	83
2	CollgCr	\$228,500	\$253,115	3	2,169	8/10	2002	81
3	Somerst	\$235,128	\$249,504	3	1,580	8/10	2007	80

Note. Listed prices are historical sale prices. The AI fair value is also computed for homes seen in training, so value gaps are somewhat optimistic; a production system would score only unseen listings.

7. Generative AI Assistant

7.1 Design

The assistant is a thin, testable layer around the Groq API (Llama 3.3 70B). Three modules cooperate: a prompts module that holds all prompt text, an LLM client that handles the API call and errors, and an assistant module that assembles messages for chat, document answers and property summaries.

7.2 Prompt Engineering

Table 9: Elements of the system prompt

Element	Purpose
Role	Defines PropMind as a professional real estate advisor and sets a concise, plain-English style
Injected market context	Real facts computed from the data (city median, highest and lowest neighborhoods, model accuracy) so answers stay grounded
Rules: no invented numbers	Figures may only come from the context or from what the user provides; otherwise the assistant says it does not know and points to the right tool
Rules: professional limits	States that it is not a lawyer, tax or financial adviser and recommends a licensed professional for major decisions
Rules: balanced advice	Mentions a key risk or trade-off, never guarantees appreciation or returns, never asks for personal identification or bank details
Rules: scope and length	Stays on real estate topics and keeps answers under about 180 words unless more detail is requested
Conversation memory	The last eight turns are sent with each request so follow-up questions work
Low temperature	0.3 for consistent, factual answers (0.1 for document answers)

7.3 Property Summaries

A separate data-only prompt asks the model for a three to four sentence summary of one property: neighborhood, size, quality, asking price against the AI fair value, one strength and one point to verify. The prompt forbids inventing features, so the summary can be trusted to reflect the data.

7.4 Reliability: Offline Mode

If no API key is configured or the call fails, the assistant does not break. Chat falls back to neighborhood facts computed from the data, document questions return the best matching passage, and property summaries use a template. The interface labels these answers as offline, so users always know what produced them.

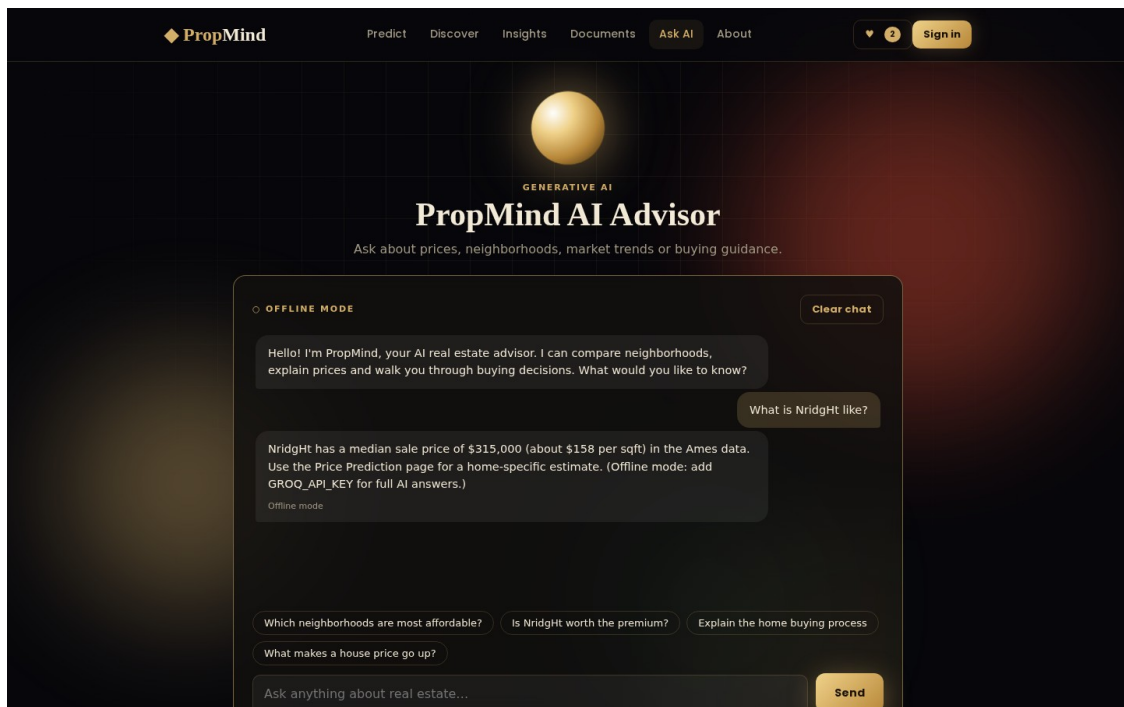


Figure 12: The AI advisor page (shown here in offline mode)

8. RAG Knowledge Assistant

8.1 Knowledge Base

The knowledge base contains four documents written for this project in the style of professional guides. Users can add their own PDF, TXT or Markdown files, which are indexed immediately.

Table 10: Documents in the knowledge base

Document	Content	Words	Passages
Ames Area Information	Neighborhood profiles generated from the dataset statistics	2,391	32
Investment Tips	Cap rate, cash-on-cash, worked example, risks, taxes	979	12
Registry & Legal Process	Iowa purchase process: offer, title, disclosures, deeds, taxes, recording	1,080	12
Property Buying Guide	Budgeting, mortgages, offers, inspection, closing, mistakes	1,134	13

8.2 Pipeline

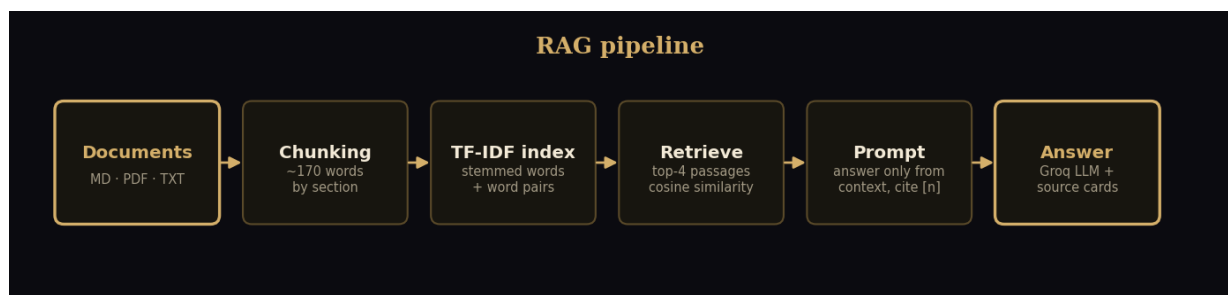


Figure 13: RAG pipeline from document to cited answer

Each document is split by section into passages of about 170 words with a short overlap. Passages are indexed with TF-IDF over lightly stemmed words and word pairs, and section titles are counted twice so that a question about "inspection" finds the section titled "Inspection and Appraisal". At question time, cosine similarity returns the top four passages.

8.3 Grounded Generation

The prompt gives the model only the numbered passages and instructs it to answer using only them, to cite passages inline as [1], [2], and to reply "I could not find this in the uploaded documents" when the passages do not contain the answer. The interface shows the answer together with the source cards (document, section, relevance score and text), so every claim can be checked.

8.4 Retrieval Evaluation

A set of 15 questions with known source documents was used to measure whether the correct document is retrieved first. The first version (plain TF-IDF) ranked the correct document first for 14 of 15 questions (93%). The one failure was "What should I check during a home inspection?", which returned a legal-process section instead of the buying guide's inspection section. Adding light stemming and title weighting fixed it:

Table 11: Retrieval accuracy on 15 evaluation questions

Retriever	Correct document ranked first	Correct document in top three
Plain TF-IDF (baseline)	14 of 15 (93%)	15 of 15 (100%)
Stemmed TF-IDF with title weighting (deployed)	15 of 15 (100%)	15 of 15 (100%)

Caveat. The evaluation set is small and was also used to guide the fix, so the 100% figure should be read as evidence that the system works on typical questions and not as a guarantee on all questions. Semantic retrieval with embeddings is listed as future work.

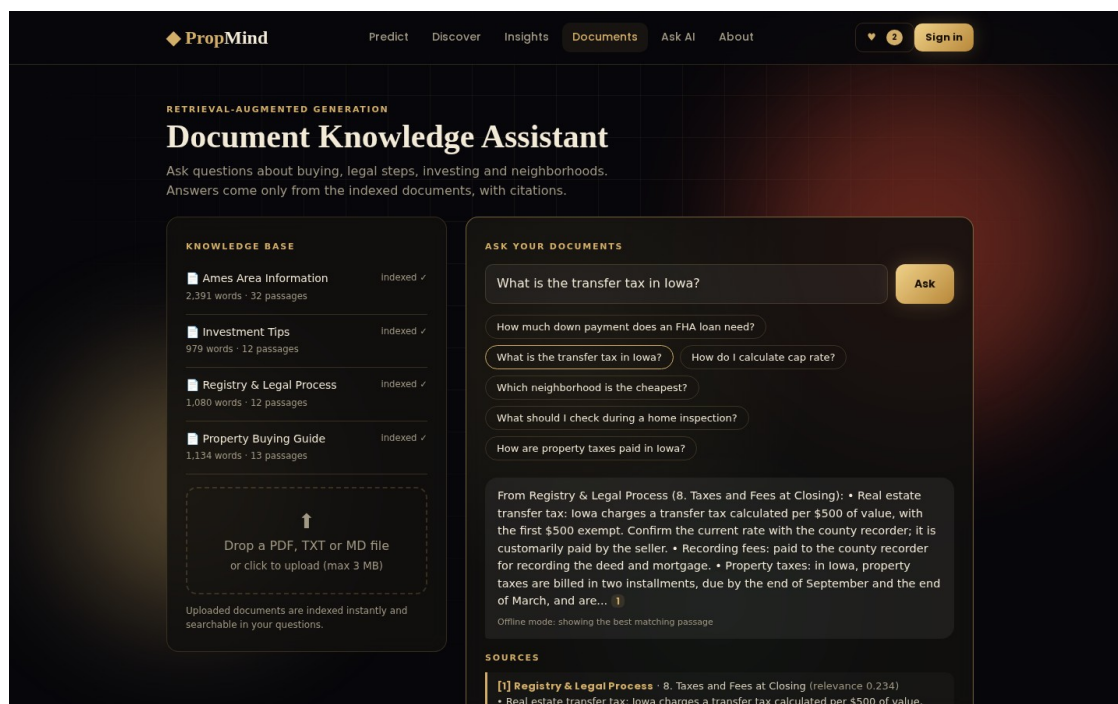


Figure 14: The document assistant answering with cited sources

9. System Design and Implementation

9.1 Architecture

The browser calls a small JSON API served by FastAPI, which also renders the seven pages with Jinja2 templates. The business logic lives in plain Python service modules that do not depend on the web framework, which keeps them easy to test and to reuse in the notebook.

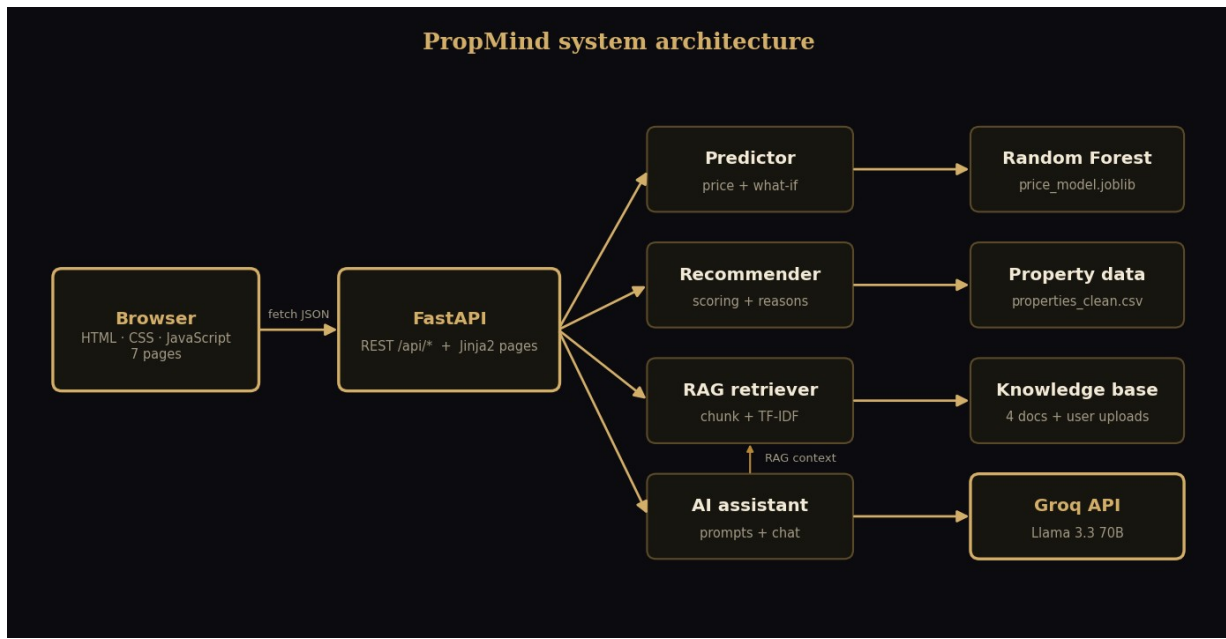


Figure 15: PropMind system architecture

9.2 Backend Modules

Table 12: Backend modules

Module	Responsibility
predictor.py	Loads the model; returns the estimate, 80% range, confidence and what-if effects
recommender.py	Filters and scores homes; builds the reasons
rag.py	Chunks documents, builds the index, retrieves passages, accepts uploads
assistant.py, llm.py, prompts.py	Prompt assembly, Groq client, offline fallbacks
main.py	Routes, input validation with Pydantic, CORS, PDF report generation

9.3 API Endpoints

Table 13: REST API

Method	Endpoint	Purpose
GET	/api/health, /api/meta	Status; neighborhoods, preferences and model metrics
POST	/api/predict	Price estimate with range and what-if
POST	/api/recommend	Ranked homes with reasons
GET	/api/properties	Property details by id
POST	/api/summary	AI summary of one property
POST	/api/chat	General AI advisor
POST	/api/rag/ask	Answer from documents with sources
GET, POST	/api/rag/docs, /api/rag/upload	List and upload documents
GET	/api/insights	Neighborhood statistics
POST	/api/report	Download a PDF valuation report

9.4 Web Interface

The interface uses a dark-luxury design system in black and gold with a serif display typeface, glass-style cards and subtle motion. It is written in plain HTML, CSS and JavaScript with no framework, which keeps the pages fast and easy to host. All seven pages are responsive and work on phones.

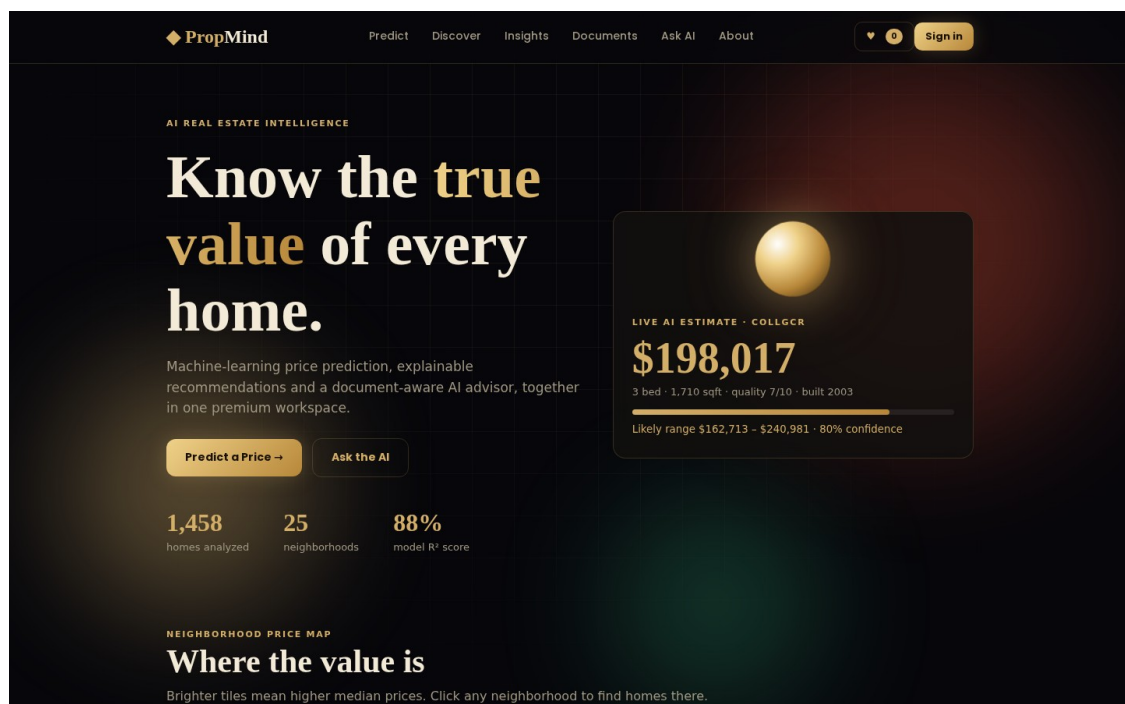


Figure 16: Home page with a live model estimate and neighborhood price map

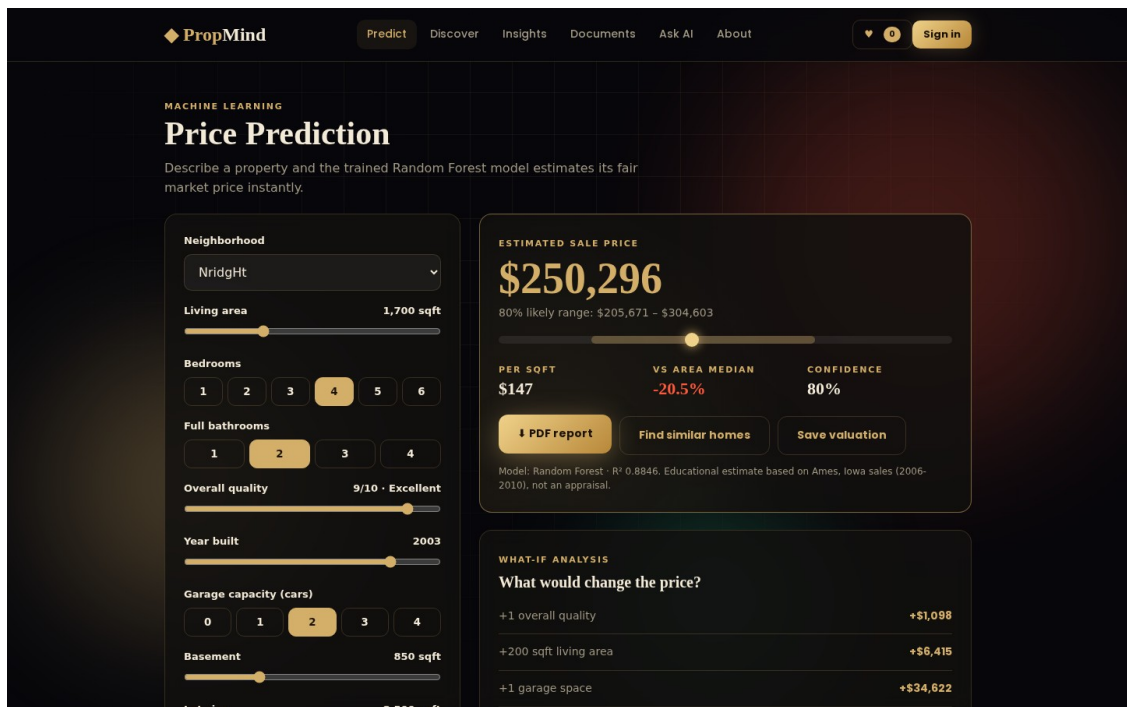


Figure 17: Price prediction page with range, what-if analysis and PDF report

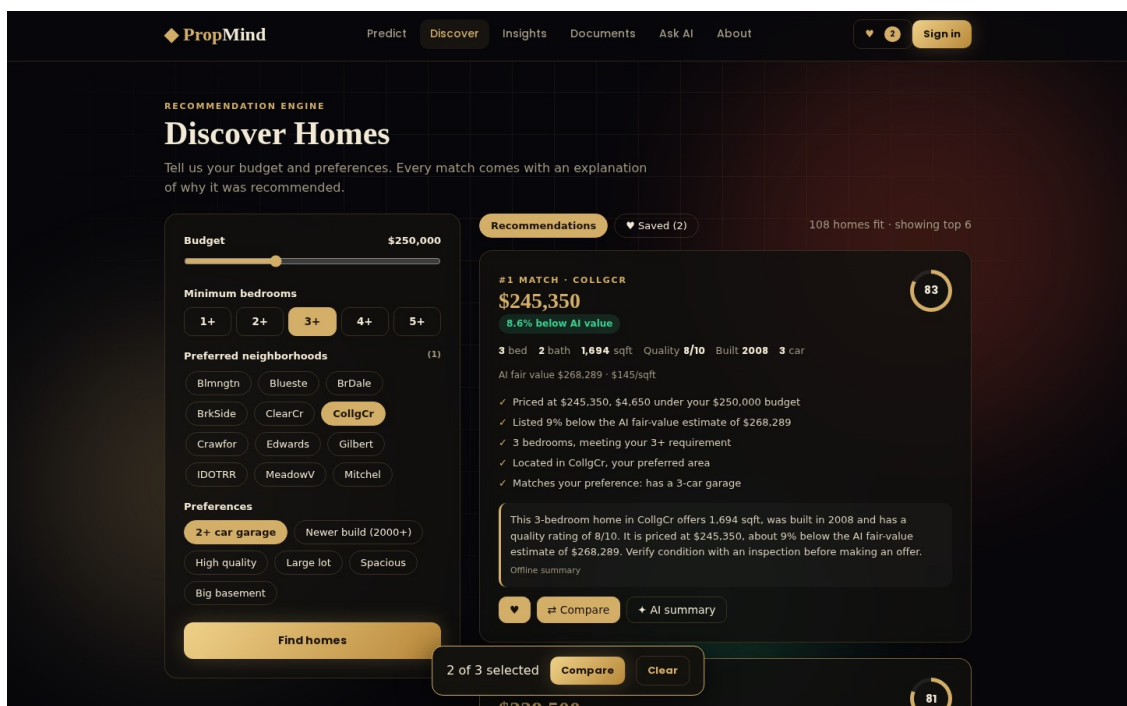


Figure 18: Recommendation page with explained matches and compare tray

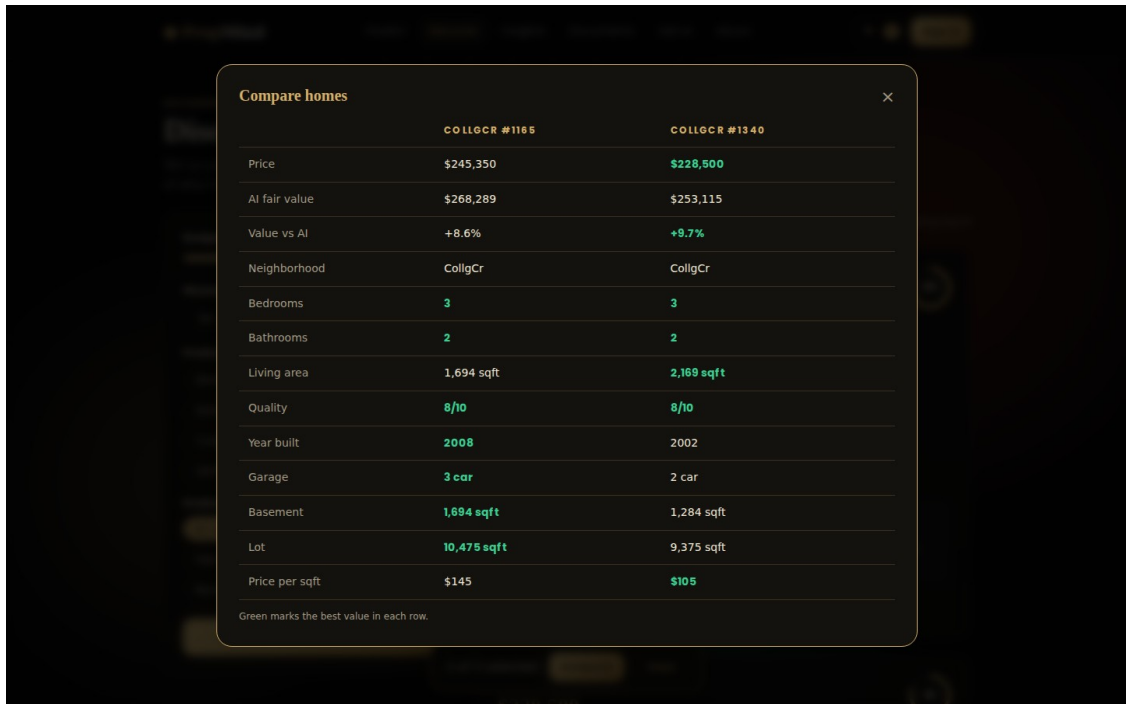


Figure 19: Side-by-side comparison of recommended homes

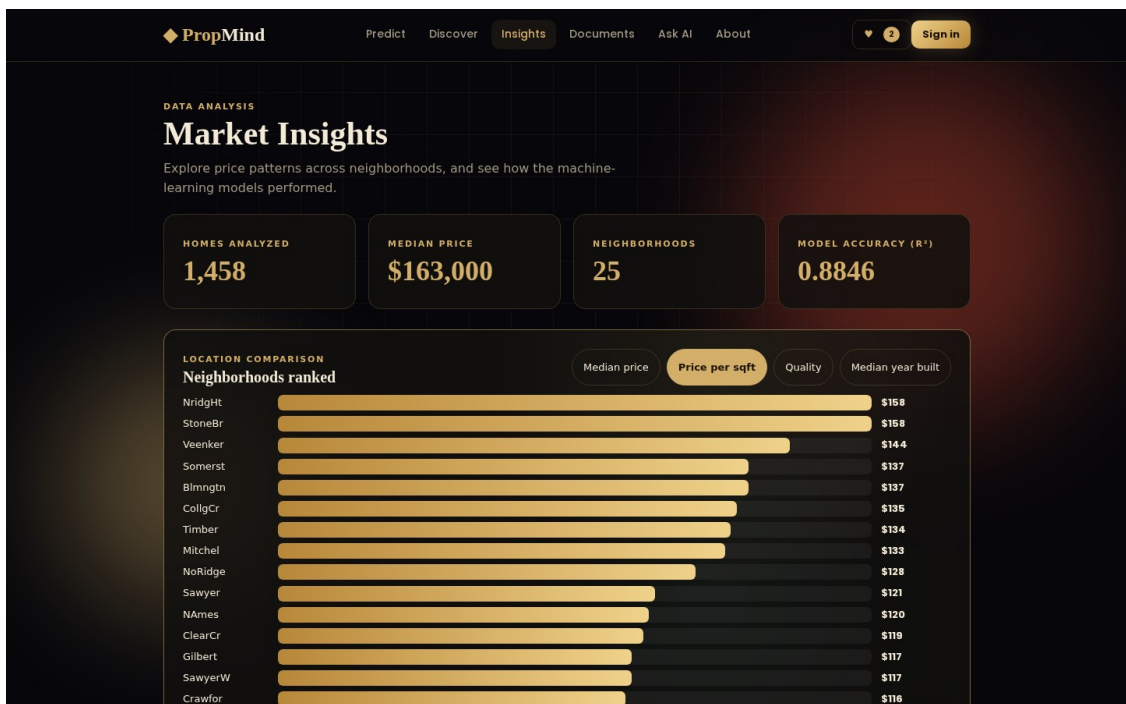


Figure 20: Market insights with interactive neighborhood ranking

9.5 Additional Features

Table 14: Features beyond the minimum requirements

Feature	Description
Login	Demo accounts stored in the browser with a hashed password; separate shortlist and history per user
Saved homes	Heart button on every result and a shortlist view
Compare	Up to three homes side by side with the best value highlighted
AI summary	One-click written summary of any property
PDF report	Downloadable valuation report
Valuation and chat history	Stored locally for the signed-in user
Offline fallback	The product keeps working without an API key
Static build	A script exports the site for Firebase Hosting

10. Testing and Quality Assurance

10.1 Automated Tests

Ten automated tests cover the core logic without needing a web server. They run with a single command and all pass.

Table 15: *Automated tests*

Test	What it checks
Prediction shape and range	Estimate is plausible and the range brackets it
Quality raises price	Quality 9 predicts more than quality 5 for the same home
Premium neighborhood costs more	Guards against the encoding defect described in Section 4.5
Recommender filters	Results respect budget, bedrooms and neighborhood and carry reasons
Recommender no-match	Impossible filters return a clear message
Property lookup	Details are returned by id
RAG retrieves the right document	Transfer tax, FHA down payment and cap rate questions find the right guide
RAG stemming	The inspection question finds the buying guide
RAG upload is searchable	A newly added document is found immediately
Assistant offline fallbacks	Chat, document answers and summaries work without a key

10.2 Browser Tests

The interface was also exercised in a real Chromium browser through Playwright, driving the application's route handlers. The tests covered the home page statistics and price map, the predictor (changing inputs, saving a valuation, downloading the PDF report), the recommendation filters, favourites, compare and AI summary, the insights charts, document questions with citations, the chat, registration and sign-in, and the mobile layout (no horizontal overflow). Console errors and failed requests were recorded during the run.

10.3 Defects Found and Fixed

Table 16: *Issues discovered during testing*

Issue	Fix
Neighborhood had almost no effect on predicted price	Replaced one-hot encoding with target encoding
RAG returned the wrong section for the inspection question	Added light stemming and title weighting
Insights bars for price per sqft and quality looked almost equal	Scaled bars from a lower baseline for those metrics
Offline document answers showed raw markdown and a repeated notice	Cleaned passages at indexing and removed the duplicate note
Training needed plotting libraries that hosting does not	Split requirements into runtime and development files

11. Deployment and Operations

The project is designed to run on free tiers without a billing card. The repository is hosted on **GitHub**. The backend and the website are deployed together on **Vercel** as a Python serverless function (configured by `vercel.json`, `index.py` and `.vercelignore`), with the Groq key stored as an environment variable. The same website can also be exported as static files with `ml/build_static.py` and published on **Firebase Hosting**, where the pages call the Vercel API. Google Cloud Run was not used because it requires billing to be enabled.

Table 17: *Deployment components*

Component	Host	Notes
Source code	GitHub	Includes data, notebook, tests and documentation
Backend and website	Vercel	Stateless function; built-in documents always available
Static frontend	Firebase Hosting	Built with <code>ml/build_static.py</code> pointing at the Vercel URL
LLM	Groq API	Key kept in the <code>GROQ_API_KEY</code> environment variable

Because serverless functions are stateless, documents uploaded through the web page may disappear between requests. The four built-in documents are part of the deployment and always remain. For permanent user documents, a database or object storage would be added (see Section 12).

12. Limitations and Future Work

12.1 Limitations

- **Data scope:** one city and 2006-2010, so estimates are educational and not appraisals.
- **Inputs:** nine features keep the form simple but ignore details such as renovation state or exact location.
- **Retrieval:** TF-IDF matches words and not meaning, so paraphrased questions can miss.
- **Recommender value gap:** uses fair values computed on homes the model has seen.
- **Accounts:** demo login lives only in the visitor's browser.
- **Documents:** legal and investment content is general and Iowa-oriented, not advice.

12.2 Future Work

- Embedding-based retrieval with a vector database for semantic search.
- Real authentication and storage with Firebase Authentication and Firestore.
- Gradient boosting models, prediction intervals from quantile regression and a larger, more recent dataset.
- Map view with listing photos and an image-based condition score.
- Multilingual interface, including Urdu.

13. Conclusion

PropMind shows how data analysis, machine learning, generative AI and retrieval can be combined into one coherent product. The final Random Forest model explains about 88% of price variance on unseen homes with an average error of \$17,782, and 77% of predictions fall within 15% of the real price. The recommender explains every match, the assistant is constrained by a careful prompt and an offline fallback, and the RAG assistant answers from real documents with citations, ranking the right document first on all 15 evaluation questions.

Just as important, testing the system as a product, and not only scoring the model, uncovered and fixed real defects such as the ignored neighborhood feature and a retrieval miss. The result is a deployable application with source code, tests, documentation and a clear path for improvement.

14. References

- [1] De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).
- [2] Kaggle. House Prices: Advanced Regression Techniques (competition and dataset). kaggle.com/c/house-prices-advanced-regression-techniques
- [3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [5] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5), 513-523.
- [6] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.
- [7] FastAPI documentation. fastapi.tiangolo.com
- [8] Groq API documentation. console.groq.com/docs

Appendix A: Setup and Run Guide

Requires Python 3.10 or newer.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-dev.txt
python ml/train.py
copy .env.example .env
uvicorn app.main:app --reload
```

Open <http://127.0.0.1:8000> for the website and <http://127.0.0.1:8000/docs> for the interactive API documentation. Add GROQ_API_KEY to the .env file to enable full AI answers; without it the assistant runs in offline mode. Run the tests with `python tests/test_core.py`.

Appendix B: Example API Call

Request to POST /api/predict:

```
{"GrLivArea": 1710, "BedroomAbvGr": 3, "FullBath": 2, "OverallQual": 7,  
  "YearBuilt": 2003, "GarageCars": 2, "TotalBsmtSF": 856, "LotArea": 8450,  
  "Neighborhood": "CollgCr"}
```

Response (abridged):

```
{"price": 198017, "low": 162713, "high": 240981,  
  "price_per_sqft": 116, "vs_neighborhood_pct": 0.4,  
  "confidence": 80, "model": "Random Forest", "r2": 0.8846,  
  "what_if": [{"change": "+1 overall quality", "effect": 17616}, ...]}
```